SESSION 4: ANSWERING BUSINESS QUESTIONS WITH DATA

A Beginner's Complete Guide to Data Analysis

WELCOME TO YOUR FIRST DAY

You're sitting at your new desk.

Your laptop is open. You have access to a database called "RetailAnalyticsDB."

You don't know what's in it yet.

Your phone buzzes. It's a Slack message from the CFO:

> "I need monthly revenue numbers for tomorrow's board meeting. Can you help?"

Your heart races. Where do you even start?

DON'T PANIC - YOU HAVE A SYSTEM

What This Session Teaches You

NOT:

- How to memorize SQL syntax
- How to be perfect on day one
- How to know everything about databases

YES:

- A step-by-step process you can follow every time
- How to figure out what data you have
- How to know if your answer is right
- How to explain the value of your work

Think of this as your GPS for data analysis. You don't need to memorize every street. You just need to know how to navigate.

THE HONEST TRUTH ABOUT DATA ANALYST JOBS

What Courses Don't Tell You

What you learned in courses:

- "Write a SELECT statement"
- "Use JOIN to combine tables"
- "GROUP BY aggregates data"

What actually happens at work:

- Someone sends you a vague request at 4 PM
- You've never seen this database before
- They need it by tomorrow morning
- You're not sure what they really want
- You don't know if your answer is correct

Today you'll learn how professionals handle this reality.

MEET YOUR IMAGINARY COMPANY

Emergi Retail: Your New Employer

Company profile:

- Australian omnichannel retailer (online + physical stores)
- Operates in NSW, VIC, QLD
- Sells electronics, home goods, outdoor equipment
- ~10,000 customers, $15M annual revenue
- Growing but facing challenges

Your role:

Junior Data Analyst

Your manager:

Head of Analytics

Your database:

RetailAnalyticsDB

You'll be answering questions from:

- CFO (finance questions)
- Marketing Director (customer questions)
- Operations Manager (product/quality questions)
- Sales Director (customer health)

THE PROFESSIONAL'S 7-STEP FRAMEWORK

Your Repeatable Process for Every Question

01

Understand the Real Business Problem

02

Clarify Success Criteria

03

Explore the Data Landscape

04

Validate Data Quality

05

Answer the Question (Write SQL)

06

Sanity Check Your Answer

07

Quantify the Business Impact

This is your GPS. Follow it every single time.

It doesn't matter if you're answering a simple question or building a complex report. These 7 steps never change.

STEP 1: UNDERSTAND THE REAL PROBLEM

People Don't Always Ask for What They Need

Example:

What they say: "I need a list of all customers."

What they might actually need:

- Active customers only (not churned)
- Customers who bought in last 90 days
- Customers with email addresses (for a campaign)
- High-value customers only
- Customers in a specific region

Your job: Figure out what they actually need BEFORE writing any SQL.

HOW TO UNCOVER THE REAL NEED

The 5 Magic Questions

Always ask these BEFORE touching your keyboard:

1

"What decision will this analysis help you make?"

Budget allocation? Marketing campaign? Product discontinuation?

2

"What's the time period you care about?"

Last month? Last year? All time? Specific dates?

3

"How will you use this information?"

One-time report? Daily dashboard? Executive presentation?

4

"What level of detail do you need?"

Individual customers? Daily totals? Monthly summaries?

5

"When do you need this by?"

30 minutes? End of day? End of week?

These questions save hours of rework.

REAL EXAMPLE: CFO REQUEST

Let's Practice Together

> **The email says:**
>
> "I need revenue numbers for the board meeting tomorrow."

What questions should you ask?

Pause and think... what's unclear about this request?

REAL EXAMPLE: QUESTIONS TO ASK

Turning Vague into Specific

Your response email:

"Happy to help! Quick clarifications:

1. **Time period**: Last 12 months, or last fiscal year, or year-to-date?
2. **Breakdown**: Do you need monthly breakdown or just total?
3. **Comparison**: Should I include year-over-year comparison?
4. **Format**: What format works best for your board deck?
5. **Additional metrics**: Just revenue, or also order counts and customer counts?"

CFO responds:

"Last 12 months, monthly breakdown, yes to YoY comparison. Excel format is fine. Include order counts."

NOW you know exactly what to build.

STEP 2: CLARIFY SUCCESS CRITERIA

What Does "Good" Look Like?

Before writing SQL, align on:

Accuracy:

How precise does this need to be?

- Board presentation = must be exact
- Exploratory analysis = approximate is fine

Completeness:

What's included vs excluded?

- All orders or just completed orders?
- All customers or just active customers?

Format:

How will they consume this?

- Email with numbers?
- Excel file?
- Dashboard?
- PowerPoint slide?

Validation:

How will you know it's correct?

- Compare to last month's report?
- Cross-check with finance's numbers?
- Does the CFO have a ballpark expectation?

STEP 3: EXPLORE THE DATA LANDSCAPE

This Is Where Beginners Skip Steps (Don't!)

The mistake beginners make:

Immediately writing SQL without understanding what data exists.

What professionals do:

Spend 10-15 minutes exploring the database structure first.

Why this matters: You can't answer questions about data you don't understand.

Think of it like cooking: You don't start chopping vegetables before checking what's in your kitchen.

DATA EXPLORATION: WHAT AM I LOOKING AT?

Question #1: What Schemas Exist?

A schema = a folder for organizing tables

In RetailAnalyticsDB, you'll see:

1

bronze

raw data (messy, all text)

2

silver

cleaned data (proper types, validated)

3

gold

analytics-ready (pre-aggregated)

4

quarantine

rejected records (data quality issues)

Think of it like organizing your closet:

- Bronze = clothes dumped on the floor
- Silver = clothes folded and sorted
- Gold = outfits ready to wear

For answering business questions, you'll usually use GOLD tables.

DATA EXPLORATION: WHAT TABLES ARE IN GOLD?

Question #2: What's Available?

In the gold schema, you'll find:

gold.CustomerMart

One row per customer

Has: TotalOrders, TotalRevenue, FirstOrderDate, LastOrderDate

gold.ProductPerformanceMart

One row per product

Has: TotalUnitsSold, TotalRevenue, ReturnRate

gold.DailySalesMart

One row per day

Has: Revenue, OrderCount, CustomerCount

gold.MonthlySalesMart

One row per month

Has: Revenue, OrderCount, AvgOrderValue

gold.CustomerRFM

One row per customer with segmentation

Has: RecencyScore, FrequencyScore, MonetaryScore, Segment

UNDERSTANDING "GRAIN" - THE MOST IMPORTANT CONCEPT

What Does "One Row Represents" Mean?

Grain = What one row in the table represents

Example 1: gold.CustomerMart

Grain: **1 row per customer**

If we have 10,000 customers → 10,000 rows

Each row summarizes ALL of that customer's activity

Example 2: gold.DailySalesMart

Grain: **1 row per day**

If we have 3 years of data → ~1,095 rows (3 years × 365 days)

Each row summarizes ALL sales for that specific day

Example 3: gold.MonthlySalesMart

Grain: **1 row per month**

If we have 3 years of data → 36 rows (3 years × 12 months)

Each row summarizes ALL sales for that specific month

Why grain matters: If you don't understand grain, you'll count things twice and get wrong answers.

GRAIN PRACTICE: WHICH TABLE SHOULD YOU USE?

Let's Practice Picking the Right Table

> **Question from CFO:** "What was our total revenue last month?"

Which table would you use?

A) gold.CustomerMart

(grain = 1 row per customer)

B) gold.DailySalesMart

(grain = 1 row per day)

C) gold.MonthlySalesMart

(grain = 1 row per month)

D) gold.ProductPerformanceMart

(grain = 1 row per product)

Pause and think...

GRAIN PRACTICE: THE ANSWER

Why C Is Correct

Answer: C) gold.MonthlySalesMart

Why?

- You need MONTHLY revenue (the grain matches what you need)
- MonthlySalesMart already has Revenue column aggregated
- You just filter to last month and grab that one number

Why not the others?

CustomerMart: You'd have to SUM revenue across all customers (extra work)

DailySalesMart: You'd have to SUM revenue across ~30 days (extra work)

ProductPerformanceMart: Wrong dimension entirely (products not time)

Pro tip: Always match the grain of your question to the grain of your table.

DATA EXPLORATION: LOOK AT ACTUAL DATA

Question #3: What's Actually in This Table?

Never trust table names alone. Look at the data.

Run this first:

```
SELECT TOP 10 *
FROM gold.CustomerMart
ORDER BY TotalRevenue DESC;
```

What you're checking:

- ✓ Column names match what you expect?
- ✓ Data types look correct? (numbers are numbers, dates are dates)
- ✓ Values seem reasonable? (no $999,999,999 revenue)
- ✓ Any obvious nulls or blanks?
- ✓ Any patterns that seem strange?

This is like inspecting ingredients before cooking.

WHAT "GOOD DATA" LOOKS LIKE

Example: gold.CustomerMart Top 10 Rows

CustomerID	CustomerName	Email	TotalOrders	TotalRevenue	LastOrderDate
CUST000042	Sarah Mitchell	sarah.m@example.com	23	$45,230.50	2024-12-15
CUST001893	James Chen	j.chen@example.com	19	$38,450.00	2024-12-20
CUST000156	Emma Thompson	emma.t@example.com	21	$35,670.25	2024-12-10
...					

What we're checking:

- ✓ CustomerID looks like a proper ID
- ✓ Names look like real names (not "Test Customer")
- ✓ Emails are formatted correctly
- ✓ Orders and Revenue are positive numbers
- ✓ LastOrderDate is recent (customers are active)

Red flags we'd worry about:

- ❌ CustomerID = "12345" (too simple)
- ❌ CustomerName = "XXXXX" (test data)
- ❌ TotalRevenue = $0 but TotalOrders = 50 (math doesn't work)
- ❌ LastOrderDate = 1900-01-01 (default date, not real)

STEP 4: VALIDATE DATA QUALITY

Trust, But Verify

Even if the table exists and has data, check quality:

Common data quality issues:

- **Nulls where they shouldn't be:** Email missing for 50% of customers?
- **Duplicates:** Same customer appears 5 times?
- **Out-of-range values:** Age = 250 years old?
- **Impossible dates:** OrderDate in the future?
- **Inconsistent totals:** Revenue doesn't match sum of orders?

How to check:

```
-- Check for nulls
SELECT
  COUNT(*) AS TotalRows,
  COUNT(Email) AS RowsWithEmail,
  COUNT(*) - COUNT(Email) AS RowsMissingEmail
FROM gold.CustomerMart;
```

What you want to see:

- TotalRows = 10,000
- RowsWithEmail = 9,800 (98% is good)
- RowsMissingEmail = 200 (2% acceptable)

Red flag:

- RowsMissingEmail = 8,000 (80% missing = problem!)

DATA QUALITY: REALISTIC EXPECTATIONS

Not All Data Is Perfect (And That's OK)

What beginners think:

"Data should be 100% perfect with no nulls or errors."

The reality:

"Data will have issues. Your job is to understand them and work around them."

Acceptable data quality:

- 95%+ completeness for key fields (CustomerID, OrderDate, Revenue)
- 85%+ completeness for optional fields (Phone, Address)
- <1% duplicates
- <1% out-of-range values

When to raise concerns:

- Key fields (CustomerID, Revenue) have >5% nulls
- Totals don't reconcile (Revenue ≠ Sum of OrderLines)
- Recent data missing (no orders in last week)

Your job: Document known issues, work around them, flag major problems.

STEP 5: ANSWER THE QUESTION

NOW You Write SQL (But Step-by-Step!)

The mistake beginners make:

Writing the entire query all at once.

What professionals do:

Build the query incrementally, validating each step.

The incremental approach:

Step 1: Get all columns, no filters

```
SELECT * FROM gold.MonthlySalesMart;
```

Look at what you got. Does it make sense?

Step 2: Add filters

```
SELECT * FROM gold.MonthlySalesMart
WHERE SalesMonth >= '2024-01-01';
```

Check: Did row count decrease? Are the months correct?

Step 3: Select only needed columns

```
SELECT SalesMonth, Revenue, TotalOrders
FROM gold.MonthlySalesMart
WHERE SalesMonth >= '2024-01-01';
```

Step 4: Add ordering

```
SELECT SalesMonth, Revenue, TotalOrders
FROM gold.MonthlySalesMart
WHERE SalesMonth >= '2024-01-01'
ORDER BY SalesMonth ASC;
```

Validate after EVERY step before proceeding.

WHY BUILD QUERIES INCREMENTALLY?

The Debugging Advantage

Scenario: Your final query returns weird results

If you wrote it all at once:

- You have to check 10 different things
- JOIN wrong? Filter wrong? Aggregation wrong? Sort wrong?
- Could take 30 minutes to find the error

If you built incrementally:

- You know exactly which step broke it
- Only have to check that one step
- Fix takes 2 minutes

Real example:

❌ **Writing all at once:**

```
SELECT c.Segment, COUNT(*), SUM(o.Revenue), AVG(o.Revenue)
FROM gold.CustomerMart c
LEFT JOIN gold.DailySalesMart o ON... -- wait, these don't JOIN!
GROUP BY c.Segment
HAVING AVG(o.Revenue) > 1000;
```

Where's the error? Hard to tell.

✓ **Building incrementally:**

```
-- Step 1: Just get customers
SELECT * FROM gold.CustomerMart; -- ✓ Works

-- Step 2: Try to join (THIS IS WHERE IT BREAKS)
SELECT c.*, o.*
FROM gold.CustomerMart c
LEFT JOIN gold.DailySalesMart o ON... -- wait, what field connects these?
--
```

STEP 6: SANITY CHECK YOUR ANSWER

Does This Make Business Sense?

After getting results, ALWAYS ask:

1

Does the number feel right?

- CFO asks for monthly revenue
- You return: $45M
- Question: "Is $45M per month realistic for our company size?"
- If annual revenue is $15M... $45M/month is WRONG

2

Can I explain this to a non-technical person?

- If you can't explain it simply, you might not understand it
- Practice: "We had $1.2M in revenue last month across 450 orders"

3

What would make this wrong?

- Did I filter correctly? (Only completed orders?)
- Did I double-count? (JOIN created duplicates?)
- Did I miss a key segment? (Forgot to include online orders?)

4

Does this align with other known facts?

- Marketing says we have 10,000 customers
- Your query returns 25,000 customers
- Something's wrong (duplicates? Wrong table?)

COMMON SANITY CHECK MISTAKES

Real Examples from Junior Analysts

Example 1: The Impossible Average

Query result: "Average order value = $5,000"

Sanity check:

- Question: "Do our customers really spend $5,000 per order?"
- Reality: We sell electronics and home goods, average order is ~$150

Diagnosis:

Probably calculated total revenue / # of products instead of / # of orders

COMMON SANITY CHECK MISTAKES

Real Examples from Junior Analysts

Example 2: The Time Traveler

Query result: "Last order date: 2099-12-31"

Sanity check:

- Question: "Did a customer order from the future?"
- Reality: Impossible

Diagnosis:

Default date value or data entry error

COMMON SANITY CHECK MISTAKES

Real Examples from Junior Analysts

Example 3: The Revenue Mismatch

Query result: "Total revenue: $50M"

Sanity check:

Question: "Finance reports $15M annual revenue, but I'm showing $50M?"

Diagnosis:

- Maybe I'm looking at 3+ years of data, not 1 year?
- Maybe I'm double-counting (bad JOIN)?
- Maybe I included refunds as positive numbers?

Always validate against known facts.

HOW TO DO A QUICK SANITY CHECK

Your Validation Checklist

After every query, run through this:

- ✓ **Row count makes sense?**
  - Expecting ~12 rows for 12 months? Got 12? ✓
  - Expecting ~12 but got 500? ❌ Something's wrong
- ✓ **Numbers are in expected range?**
  - Revenue between $100K-$2M per month? ✓
  - Revenue suddenly = $0 or $999M? ❌ Investigate
- ✓ **Dates make sense?**
  - All dates in the past? ✓
  - Dates in the future or year 1900? ❌ Data issue
- ✓ **Nulls where expected?**
  - Some customers missing phone numbers? ✓ (Optional field)
  - All customers missing CustomerID? ❌ Critical problem
- ✓ **Trends make sense?**
  - December higher than February? ✓ (Holiday season)
  - Every month exactly the same revenue? ❌ Suspicious

STEP 7: QUANTIFY THE BUSINESS IMPACT

Why Should Anyone Care About Your Work?

The difference between junior and senior analysts:

Junior analyst:

"Here's the list of at-risk customers you asked for."

Sends spreadsheet, waits for next request

Senior analyst:

"Here are 237 at-risk customers representing $450K in historical revenue. If we win back 15% through proactive outreach, that's $67K recovered. The email campaign costs $5K to run. ROI is 13x. Want me to start segmenting them by value?"

Drives the business decision forward

You want to be the senior analyst.

HOW TO CALCULATE ROI

The Simple Formula

ROI = (Value Created - Cost) / Cost × 100%

Example 1: Automated Reporting

Before your work:

- Finance analyst spends 2 hours every month manually building report in Excel
- Hourly rate: $75/hour
- Cost: 2 hours × $75 × 12 months = **$1,800/year**

After your work:

- You build SQL query that runs in 30 seconds
- Finance clicks button, gets report
- Time saved: **$1,800/year**

Your ROI:

Time spent building query: 1 hour = $75

ROI: ($1,800 - $75) / $75 = 2,300%

HOW TO CALCULATE ROI (CONTINUED)

Example 2: Identifying At-Risk Customers

Business context:

- You identify 500 "at-risk" customers
- Average historical value per customer: $2,000
- Total at-risk revenue: $1M

If we do nothing:

- Historical churn rate for at-risk segment: 30%
- Lost customers: 150
- **Lost revenue: $300K**

If we do proactive outreach:

- Win-back rate through email campaign: 15%
- Saved customers: 75
- Saved revenue: $150K
- Cost of campaign: $10K
- **Net value: $140K**
- **ROI: ($150K - $10K) / $10K = 1,400%**

Your analysis just justified a $140K business decision.

QUANTIFYING IMPACT: MORE EXAMPLES

Practice Thinking in Business Terms

Example 3: Product Quality Issues

Your analysis:

- Found 12 products with >20% return rate
- These products generate $200K/year in sales
- Returns cost $25 each to process
- Current return volume: 1,200 units/year
- Current cost: **$30K/year in returns**

Business action:

- Operations fixes quality issues
- Return rate drops from 20% to 10%
- Returns saved: 600 units/year
- **Savings: $15K/year**

Your work created a business case for $15K in operational improvements.

QUANTIFYING IMPACT: MORE EXAMPLES

Practice Thinking in Business Terms

Example 4: Campaign Timing Optimization

Your analysis:

- Marketing runs email campaigns every month
- You discover Tuesday-Thursday emails get 2x open rates vs Monday/Friday
- Campaign cost: $50K per month
- Current conversion rate: 2%
- Improved conversion rate: 4% (from better timing)

Impact:

- Monthly revenue from campaigns: $500K
- Improved revenue: $1M
- **Additional revenue: $500K/year from timing alone**

Zero additional cost. Pure optimization value.

NOW LET'S SEE THIS IN ACTION

We'll Walk Through Real Scenarios Together

For each scenario, we'll follow all 7 steps:

01
Understand the real problem

02
Clarify success criteria

03
Explore the data

04
Validate data quality

05
Write SQL (incrementally!)

06
Sanity check the answer

07
Quantify the impact

You'll see exactly how professionals think through business questions.

Get ready to open SQL Server Management Studio.

SCENARIO 1: FINANCE DASHBOARD

The Request

FROM: Sarah Chen, CFO

TO: Data Analytics Team

SUBJECT: Urgent - Board Meeting Tomorrow Morning

"Hi team,

Board meeting is at 9 AM tomorrow. I need monthly revenue figures for the last 12 months to show our growth trajectory. The board will ask questions, so I also need order counts to explain any unusual months.

Can you have this ready by end of day today?

Thanks, Sarah"

Time: 2:00 PM (You have 3 hours)

SCENARIO 1: STEP 1 – UNDERSTAND THE PROBLEM

What's Sarah Really Asking For?

Surface request:

"Monthly revenue for last 12 months"

Deeper questions:

What decision is being made?

- Board is assessing company health
- Deciding whether to invest in growth or cut costs
- Sarah needs to defend company performance

What does success look like?

- Clear upward trend = good (board happy)
- Flat or down trend = needs explanation
- She needs to tell a story with these numbers

Why does she need order counts?

- If revenue drops but order count rises = lower average order value (needs explanation)
- If both drop = losing customers (bigger problem)

What's her real need? A presentation-ready summary that helps her confidently answer board questions.

SCENARIO 1: STEP 2 - CLARIFY SUCCESS CRITERIA

What You'd Actually Ask Sarah (In Real Life)

Your response email:

"Hi Sarah,

Absolutely, I'll get this ready. Quick clarifications:

1. **Format**: Would you like this in Excel so you can drop it into your board deck?
2. **Additional context**: Should I include year-over-year comparison if we have prior year data?
3. **Order counts**: Per month, correct?
4. **Completed orders only**: Or all orders including cancelled?
5. **Validation**: Do you have a ballpark expectation for annual revenue so I can validate my numbers?

I'll have this to you by 4 PM.

Thanks!"

Sarah responds:

"Excel works great. Yes to YoY if available. Completed orders only. We did about $15M last year total. Thanks!"

NOW you know exactly what to build.

SCENARIO 1: STEP 3 – EXPLORE THE DATA

Which Table Has Monthly Revenue?

You open SQL Server and think through your options:

Option A: gold.DailySalesMart

- Grain: 1 row per day
- Problem: Would need to SUM 30 days per month (extra work)

Option B: gold.MonthlySalesMart

- Grain: 1 row per month ✓
- Has: Revenue, TotalOrders, SalesMonth ✓
- **This is the perfect match!**

Option C: gold.CustomerMart

- Grain: 1 row per customer
- Problem: Wrong grain entirely

Decision: Use gold.MonthlySalesMart

SCENARIO 1: STEP 4 - VALIDATE DATA QUALITY

Before Using the Table, Check It

Query 1: What's in this table?

```
SELECT TOP 5 *
FROM gold.MonthlySalesMart
ORDER BY SalesMonth DESC;
```

What you see:

Sales Month	Year	Month	MonthName	Revenue	TotalOrders	TotalCustomers
2024-12-01	2024	12	December	$1,450,230	523	312
2024-11-01	2024	11	November	$1,320,450	489	298
2024-10-01	2024	10	October	$1,100,890	445	276
...						

Validation checks:

- ✓ SalesMonth is first day of each month (good - consistent grain)
- ✓ Revenue and TotalOrders are positive numbers
- ✓ Recent months show recent dates
- ✓ MonthName matches Month number

Looks good! Safe to use.

SCENARIO 1: STEP 5 - WRITE SQL (INCREMENTALLY)

Building the Query Step-by-Step

Step 1: Get all months, see what's there

```
SELECT *
FROM gold.MonthlySalesMart
ORDER BY SalesMonth DESC;
```

Check: Do you see months of data? ✓

SCENARIO 1: STEP 5 - WRITE SQL (INCREMENTALLY)

Building the Query Step-by-Step

Step 2: Filter to last 12 months

```
SELECT *
FROM gold.MonthlySalesMart
WHERE SalesMonth >= DATEADD(MONTH, -12, GETDATE())
ORDER BY SalesMonth DESC;
```

Check: Do you see exactly 12 rows (or fewer if business is new)? ✓

SCENARIO 1: STEP 5 - WRITE SQL (INCREMENTALLY)

Building the Query Step-by-Step

Step 3: Select only columns CFO needs

```
SELECT
  SalesMonth,
  MonthName,
  Revenue,
  TotalOrders
FROM gold.MonthlySalesMart
WHERE SalesMonth >= DATEADD(MONTH, -12, GETDATE())
ORDER BY SalesMonth ASC; -- Oldest first for board presentation
```

Check: Are columns what Sarah asked for? ✓

SCENARIO 1: WHAT YOU SEE

The Results

SalesMonth	MonthName	Revenue	TotalOrders
2024-01-01	January	$1,050,230	412
2024-02-01	February	$980,450	385
2024-03-01	March	$1,120,890	438
2024-04-01	April	$1,080,670	425
2024-05-01	May	$1,190,340	456
2024-06-01	June	$1,350,120	512
2024-07-01	July	$1,100,560	441
2024-08-01	August	$1,150,780	448
2024-09-01	September	$1,180,920	459
2024-10-01	October	$1,200,450	468
2024-11-01	November	$1,420,680	534
2024-12-01	December	$1,550,340	598

Total: ~$14.4M

← EOFY spike (June), ← Black Friday (November), ← Christmas (December)

SCENARIO 1: STEP 6 – SANITY CHECK

Does This Make Business Sense?

Check #1: Does total match expectations?

- Sarah said "about $15M last year"
- We're showing $14.4M for the trailing 12 months
- **✓ Close enough (within 5%)**

Check #2: Do the patterns make sense?

- June spike (EOFY in Australia) ✓
- November spike (Black Friday) ✓
- December highest (Christmas) ✓
- February lowest (post-holiday slump) ✓

Check #3: Are order counts reasonable?

- Average: ~460 orders/month
- Average order value: ~$2,500-$3,000 per order
- **Wait... that seems high for retail!**

SCENARIO 1: SANITY CHECK - CATCHING AN ERROR

Something Doesn't Add Up

The math:

$1,050,230 / 412 orders = **$2,549 per order**

Reality check:

- We sell electronics and home goods
- Typical retail order: $100-$500
- $2,500 seems very high

What could be wrong?

Option 1: Maybe we're a high-end retailer? (Check with Sarah)

Option 2: Maybe I'm looking at B2B orders, not retail? (Check data)

Option 3: Maybe there's a data quality issue? (Investigate)

What to do: Email Sarah: "The numbers look reasonable, but average order value is ~$2,500. Is that in line with your expectations for our business?"

Sarah responds: "Oh good catch! We're including both retail and wholesale. Wholesale orders are much larger. That's correct."

Crisis averted by sanity checking!

SCENARIO 1: STEP 7 - QUANTIFY THE IMPACT

Why Your Work Matters

What you delivered:

- Clean, presentation-ready monthly revenue summary
- Delivered in 1 hour

What this replaces:

Before you:

- Finance analyst pulls data from 3 different systems
- Manually builds Excel pivot tables
- Cross-references with accounting
- Takes 2-3 hours every month
- Risk of manual errors

After you:

- Run SQL query (30 seconds)
- Export to Excel (30 seconds)
- Done.

Impact:

- Time saved: 2.5 hours per month
- Annual savings: 30 hours/year
- Finance analyst hourly rate: $75/hour
- **Annual value: $2,250**

Plus:

- Board meeting goes smoothly
- CFO confidently explains trends
- Company projects credibility to board
- **Intangible value: Priceless**

SCENARIO 1: COMPLETE

What You Just Learned

You followed the 7-step framework:

- [x] **✓ Step 1:**

  Understood Sarah needs board-ready numbers, not just data

- [x] **✓ Step 2:**

  Clarified format, time period, validation expectations

- [x] **✓ Step 3:**

  Chose right table (MonthlySalesMart over DailySalesMart)

- [x] **✓ Step 4:**

  Validated table quality before using it

- [x] **✓ Step 5:**

  Built query incrementally, checking each step

- [x] **✓ Step 6:**

  Caught potential issue with average order value

- [x] **✓ Step 7:**

  Quantified $2,250 annual value created

This framework works for EVERY business question.

Now let's do another scenario, and you'll see the pattern repeat.

SCENARIO 2: VIP CUSTOMER LIST

The Request

FROM: James Wu, Marketing Director

TO: Data Analytics Team

SUBJECT: Exclusive Product Launch Event

"Hey team,

We're hosting an exclusive VIP product launch event next month. Very high-touch - we're hand-writing invitations and doing personalized outreach.

I need our absolute top customers. Maybe top 50?

I need their names, emails, and how much they've spent with us. We're going to make them feel special.

Budget for this event is $50K, so we need to make sure we're inviting the RIGHT people.

Thanks!"

SCENARIO 2: STEP 1 - UNDERSTAND THE PROBLEM

What's James Really Asking For?

Surface request:

"Top 50 customers"

Deeper questions to ask yourself:

What decision is being made?

- $50K budget event (high stakes)
- Personal invitations (white-glove treatment)
- Goal: Make VIPs feel valued, drive loyalty

What does "top" mean?

- Highest lifetime revenue?
- Most recent purchases?
- Highest order frequency?
- All of the above?

What are the risks?

- Inviting inactive customers (they won't come)
- Missing active high-value customers (they'll feel snubbed)
- Inviting wrong segment (doesn't match brand)

What's the real need? A list of truly engaged, high-value customers who will attend and appreciate the gesture.

SCENARIO 2: STEP 2 - CLARIFY SUCCESS

Questions You'd Ask James

Your response:

"Love this idea! Quick questions:

1. **Top 50 by lifetime revenue?** Or should I factor in recency (exclude customers who haven't bought recently)?
2. **Any minimum criteria?** Like must have ordered in last 6 months?
3. **Geographic focus?** Or all regions?
4. **Include their phone numbers?** For personal invitation calls?
5. **Want me to flag any at-risk VIPs?** (High value but haven't ordered recently)

Want to make sure you're inviting people who will actually come!"

James responds:

"Great questions! Top 50 by lifetime revenue. Include only active customers (ordered in last 90 days). All regions fine. Yes to phones. And definitely flag any VIPs who are at risk - good thinking!"

SCENARIO 2: STEP 3 - EXPLORE THE DATA

Which Table For Customer Data?

Option A: gold.CustomerMart

- Grain: 1 row per customer ✓
- Has: TotalRevenue ✓
- Has: CustomerName, Email ✓
- Has: DaysSinceLastOrder ✓ (for recency check)
- **Perfect match!**

Option B: gold.CustomerRFM

- Also has customer data
- Has segments (Champions, Loyal, etc.)
- Could be useful for validation

Decision: Use CustomerMart, validate with RFM segments

SCENARIO 2: STEP 4 - VALIDATE DATA

Check CustomerMart Quality

Query:

```
SELECT TOP 10 *
FROM gold.CustomerMart
ORDER BY TotalRevenue DESC;
```

What you see:

CustomerID	Customer Name	Email	Total Revenue	TotalOrders	Days Since Last Order	Segment
CUST00042	Sarah Mitchell	sarah.m@example.com	$45,230	23	5	Champions
CUST01893	James Chen	j.chen@example.com	$38,450	19	12	Champions
CUST00156	Emma Thompson	emma.t@example.com	$35,670	21	3	Champions

Validation:

- ✓ Names look real (not "Test Customer")
- ✓ Emails properly formatted
- ✓ Revenue values reasonable
- ✓ DaysSinceLastOrder shows recent activity
- ✓ Segment aligns (top customers are "Champions")

Looks good!

SCENARIO 2: STEP 5 - WRITE SQL

Building Incrementally

Step 1: Get top customers by revenue

```
SELECT TOP 50
  CustomerID,
  CustomerName,
  Email,
  TotalRevenue,
  TotalOrders
FROM gold.CustomerMart
ORDER BY TotalRevenue DESC;
```

Check: Do you see 50 rows? ✓

SCENARIO 2: STEP 5 - WRITE SQL

Building Incrementally

Step 2: Add recency filter (active in last 90 days)

```
SELECT TOP 50
  CustomerID,
  CustomerName,
  Email,
  TotalRevenue,
  TotalOrders,
  DaysSinceLastOrder
FROM gold.CustomerMart
WHERE DaysSinceLastOrder <= 90  -- Active in last 90 days
ORDER BY TotalRevenue DESC;
```

Check:

- Do you see fewer than 50 rows? (Some high-value customers might be inactive)
- Are all DaysSinceLastOrder ≤ 90? ✓

SCENARIO 2: STEP 5 - WRITE SQL

Building Incrementally

Step 3: Add segment for validation + flag at-risk

```
SELECT TOP 50
 CustomerID,
 CustomerName,
 Email,
 TotalRevenue,
 TotalOrders,
 DaysSinceLastOrder,
 Segment,
 CASE
 WHEN DaysSinceLastOrder > 60 THEN '
```

SCENARIO 2: WHAT YOU SEE

The VIP List

CustomerID	CustomerName	Email	TotalRevenue	DaysSince	Segment	ActivityStatus
CUST00042	Sarah Mitchell	sarah.m@ex.com	$45,230	5	Champions	Active
CUST01893	James Chen	j.chen@ex.com	$38,450	12	Champions	Active
CUST00156	Emma Thompson	emma.t@ex.com	$35,670	3	Champions	Active
...	...	...	...	...	...	...
CUST03421	David Park	d.park@ex.com	$28,340	75	Champions	⚠️ At Risk (60-90d)
...	...	...	...	...	...	...
CUST08392	Lisa Rodriguez	l.rod@ex.com	$25,120	82	Loyal	⚠️ At Risk (60-90d)

SCENARIO 2: STEP 6 - SANITY CHECK

Does This List Make Sense?

Check #1: Revenue ranges

- Top customer: $45K lifetime value
- #50 customer: ~$25K lifetime value
- **Question:** Is $25K realistic for a VIP threshold?
- **Answer:** For $50K event budget = $1K/guest, YES makes sense ✓

Check #2: Segments align

- Most are "Champions" or "Loyal Customers" ✓
- None are "Churned" or "At Risk" ✓ (because we filtered to last 90 days)

Check #3: At-Risk flags

- 8 customers are 60-90 days since last order
- **Action:** Tell James these might need extra personal outreach

Check #4: Missing anyone obvious?

Run quick check: Are there customers >90 days but with VERY high value?

```
SELECT TOP 10
  CustomerName,
  TotalRevenue,
  DaysSinceLastOrder
FROM gold.CustomerMart
WHERE
DaysSinceLastOrder > 90
ORDER BY TotalRevenue
DESC;
```

Result: Yes, 3 customers with >$40K value haven't ordered in 120+ days

Action: Flag for James - these are churn risks worth saving

SCENARIO 2: STEP 7 – QUANTIFY IMPACT

Event Economics

What James is planning:

- 50 VIP invitations
- Expected attendance: 30 customers (60% acceptance rate)
- Event cost: $50K
- Cost per attendee: $1,667

Historical data from your query:

- Top 50 customers = $1.5M lifetime value (sum of TotalRevenue)
- Average per customer: $30K

Expected event outcomes:

- Cross-sell new product to 30 attendees
- Conversion rate: 40% (12 customers)
- Average purchase: $2,500
- **Event revenue: $30K**

Plus retention value:

- VIPs who feel valued are 2x more likely to make next purchase
- Average annual spend: $3K per VIP
- 30 attendees × 50% retention uplift × $3K = **$45K/year**

Total value: $30K (immediate) + $45K (retention) = $75K

Cost: $50K

ROI: 50% (plus brand value and loyalty)

Your analysis ensures the guest list justifies the investment.

SEE THE PATTERN?

The Framework Repeats

Every scenario follows the same 7 steps:

01
Understand the real problem

02
Clarify success criteria

03
Explore available data

04
Validate data quality

05
Write SQL incrementally

06
Sanity check results

07
Quantify business impact

This is your superpower.

Let's keep going. Each scenario makes this framework more automatic.

WHAT YOU'RE LEARNING

Skills Beyond SQL

Technical skills (what courses teach):

- Writing SELECT statements ✓
- Using JOINs ✓
- Filtering with WHERE ✓

Professional skills (what makes you valuable):

- Asking clarifying questions before coding
- Understanding table grain and relationships
- Building queries incrementally
- Validating results for business sense
- Explaining ROI in business terms
- Thinking like a consultant, not a coder

Both matter. But the second list is what gets you hired and promoted.